1. Analyze the two GenAI programs. The analysis must include:
   a. Name of two GenAI programs
   Claude
   ChatGPT

b. How you accessed them (e.g., Google query, CoPilot on my computer)
Both through google so the website for both of the AI's

c. Prompt used. (You must use the same prompt for both GenAIs.)
I used 2 - 3 images of the assignment requirements on the second page and then the test file format and the sample text.

d. Generated code from both GenAIs.
I cant paste both here because they are both about 500 lines of code and comments, but they are in the repo and easy to access if you have this pdf

e. Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the “GenAI Code Comparison” lecture.

Claude

1. Correctness - The program correctly implements a Sudoku solver using object-oriented programming and recursive backtracking. It uses two classes, Board and SudokuSolver, to separate puzzle management from solving. The program checks for duplicate numbers in rows, columns, and 3×3 blocks using bitmasks. It also validates the input files and uses recursion to find all possible solutions instead of stopping at the first one. If no solution exists, it prints "No solution found.".
2. Execution Time - The program has a worst-case time complexity of O(81 × 9^k), where k represents the number of empty cells. This is because each empty cell can potentially have nine possible values, and the program scans the 81-cell board during each recursive call.However, the program improves execution time by using bitmasks to check valid numbers in constant time, rather than repeatedly searching rows, columns, and blocks. It also uses the most-constrained-cell strategy, selecting the empty cell with the fewest possible numbers.
3. Space Complexity - The program uses a fixed 9×9 array to store the Sudoku board, along with three arrays of bitmasks for rows, columns, and blocks. Since these arrays have fixed sizes, they require O(1) space. Recursive backtracking requires an additional O(k) space for the call stack, where k is the number of empty cells. However, the program also stores every solution in a vector, requiring O(S) additional space, where S is the number of

solutions found, because each stored board has a fixed size. Therefore, the overall space complexity is O(k + S).

4. Maintainability - The program is maintainable because it uses an object-oriented structure with separate Board and SudokuSolver classes. The Board class handles loading, validating, storing, and printing puzzles, while the SudokuSolver class handles recursive backtracking. This separation makes the code easier to understand, debug, and modify. The program also includes a detailed prologue, comments explaining major sections, and comments describing individual lines.

ChatGPT

1. Correctness - The program correctly implements a Sudoku solver using recursive depth-first search with backtracking. It checks that each number from 1 to 9 appears only once in each row, column, and 3×3 block. The program also validates the puzzle files before attempting to solve them. If a number cannot be placed, the program backtracks and tries another possibility. It can find multiple solutions and prints "No solution found" when a puzzle cannot be solved.
2. Execution Time - The program has a worst-case exponential time complexity of approximately O(9^k), where k is the number of empty cells. This is because each empty cell can potentially have up to nine possible numbers. However, the program improves execution time by selecting the empty cell with the fewest valid candidates first. This reduces unnecessary searching and allows the program to identify invalid paths earlier.
3. Space Complexity - The program uses a fixed 9×9 array to store the Sudoku puzzle, requiring O(1) space because the board size never changes. However, recursive backtracking uses additional memory for the function call stack. In the worst case, the recursion depth is equal to the number of empty cells, giving an auxiliary space complexity of O(k). Since a Sudoku puzzle contains at most 81 cells, the total memory usage is relatively small.
4. Maintainability - The program is maintainable because it uses an object-oriented design with a SudokuSolver class. The code separates different responsibilities into methods such as load(), canPlace(), search(), printBoard(), and solve(). This makes the program easier to understand, debug, and modify. The code also includes a prologue, comments explaining major sections, and comments describing individual lines.

f. Select one to use as the basis for this assignment and justify why you picked it.
claude because its two class design makes the maintainability easier and the bitmasks makes checking valid numbers efficient.

g. Explain what you did to improve it in terms of correctness, execution time, space complexity, and maintainability.

1. Correctness - The improved program correctly implements a Sudoku solver using object-oriented programming, recursive backtracking, and bitmasks. It checks that numbers do not repeat in any row, column, or 3×3 block. The program improves correctness by validating that each input file contains exactly 81 cells and rejecting invalid characters, extra data, and conflicting starting numbers. It also supports multiple input formats, including spaces between numbers and zeros or underscores for empty cells. The program finds all possible solutions and prints "No solution found" if none exist. Overall, these changes make the program more reliable and better at handling invalid inputs.
2. Execution Time - The program still has a worst-case exponential time complexity of approximately O(k × 9^k), where k represents the number of empty cells. However, several improvements make it faster in practice. Instead of scanning all 81 cells during every recursive call, the program creates a list of empty cells and searches only those positions. It also uses bitmasks to check valid numbers efficiently and selects the cell with the fewest candidates. Another improvement is that the program stops searching for a better cell when it finds one with only one possible number. This reduces unnecessary comparisons and recursive work.
3. Space Complexity - The program uses a fixed 9×9 array and three bitmask arrays, which require O(1) space. It also stores a list of empty cells, requiring O(k) space, and recursive backtracking uses an additional O(k) space for the call stack. Therefore, the auxiliary space complexity is O(k). A major improvement over the previous version is that the program no longer stores every solution in a vector. Instead, it prints each solution immediately using a callback function. This eliminates the O(S) memory requirement for storing S solutions, making the program more memory-efficient.
4. Maintainability - The improved program is easier to maintain because it uses named constants such as SIZE, BOX, and ALL_DIGITS instead of repeatedly using hardcoded values. It maintains separate Board and SudokuSolver classes, keeping puzzle management and solving responsibilities organized. The program also replaces the compiler-specific builtin_popcount() function with the more portable std::bitset::count(), improving compatibility across compilers. Detailed comments explain the purpose of each method and major section.

Command lines to compile and run programs if needed

ChatGPT: g++ -std=c++17 Assignment4chatgpt.cpp -o Assignment4chatgpt && ./Assignment4chatgpt

Claude: g++ -std=c++17 Assignment4claude.cpp -o Assignment4claude && ./Assignment4claude

Improved Claude: g++ -std=c++11 -Wall -Wextra -pedantic Assignment4improvedclaude.cpp -o Assignment4improvedclaude && ./Assignment4improvedclaude